

COMPX304-23A

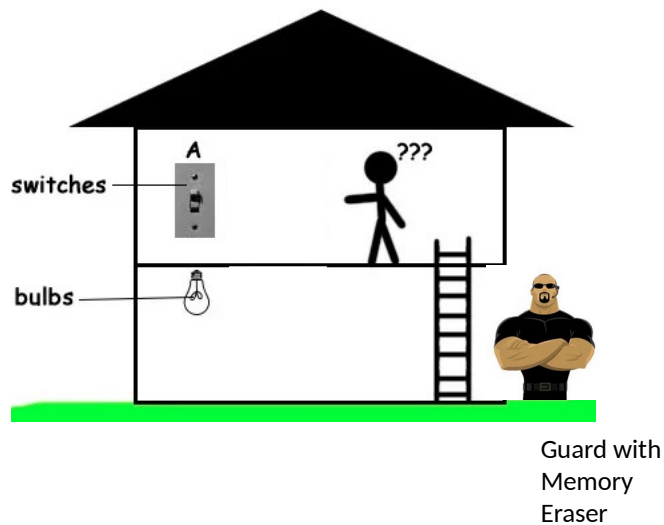
Advanced Networking and Cybersecurity

Week 11
Hardware Vulnerabilities

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros





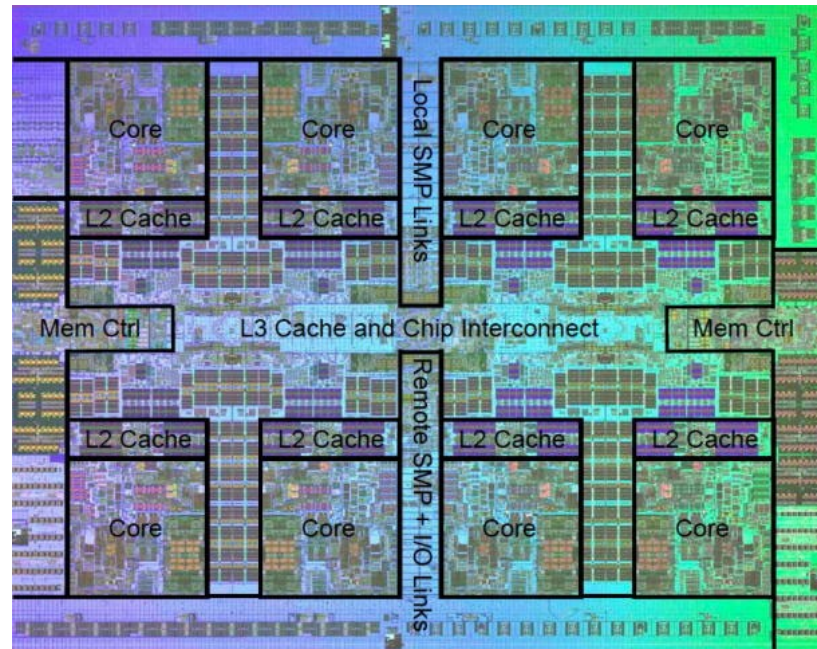
Side Channel attacks

- In channel attacks
 - Attacker uses the specified communication channel for attack
- Side channel attacks
 - The attacker uses unspecified input and output channels to convey system information
 - Examples
 - Timing
 - Disk Usage
 - Memory Usage
 - Sound
 - Power consumption
 - Cache Usage



CPU Cache

- A load 0x... is not just a load!
 1. The address is converted to its physical address and permissions are checked
 2. The address is looked up in the Level-1 (L1) cache. If not there,
 3. It's looked up in the L2 cache. If not there,
 4. In the L3 cache, if not there
 5. From main memory (cache miss)
- Each of these pathways take a significantly different time (CPU cycles) to complete

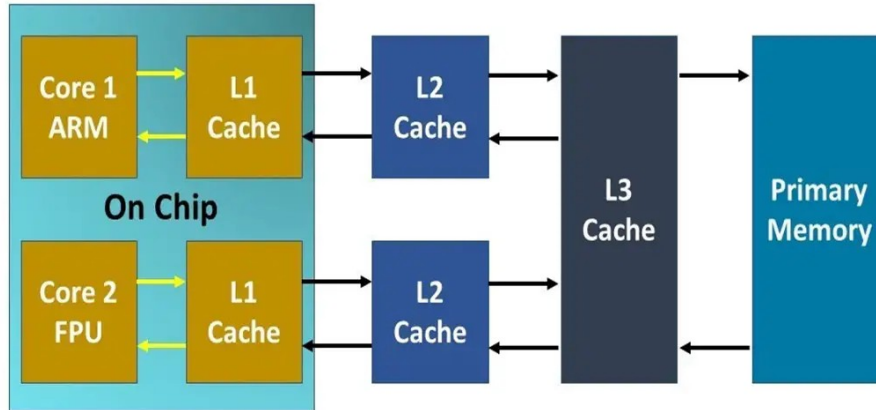


<https://arstechnica.com/gadgets/2009/09/ibms-8-core-power7-twice-the-muscle-half-the-transistors/>

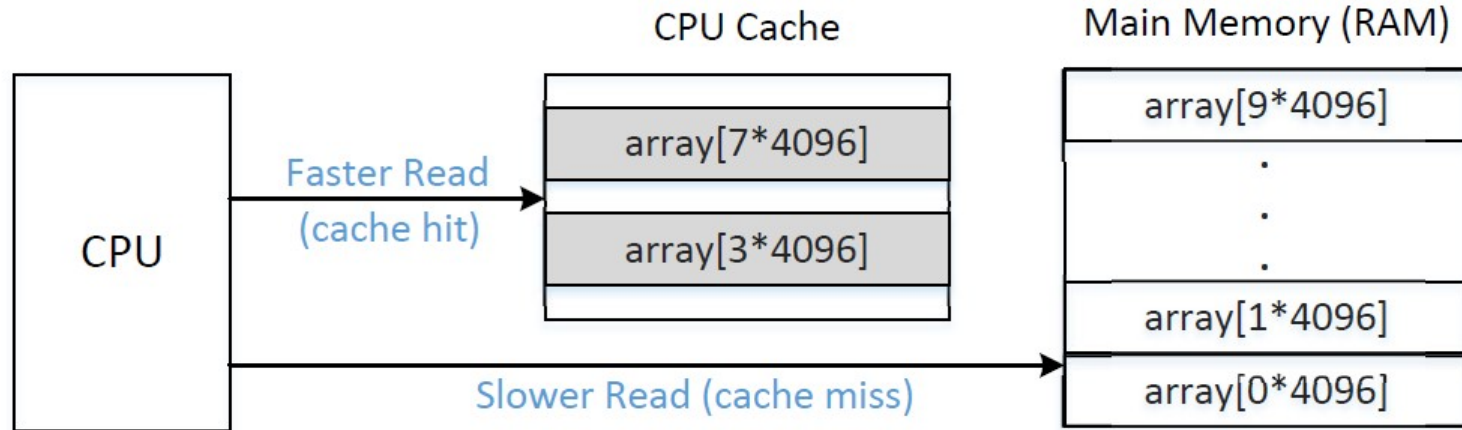


CPU Cache

Cache Memory



CPU Cache



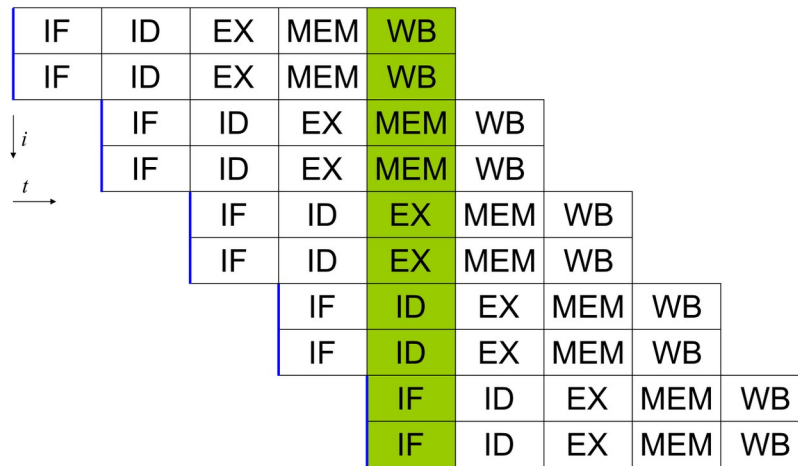
CPU Instructions (ISA)

```
01 .MODEL SMALL
02 .STACK 100H
03 .CODE
04
05 MOV AX, 0x3C
06 MOV BX, 00000000000001010B
07 ADD AX, BX
08 MOV BX, 14
09 SUB AX, BX
10
11 MOV AH, 04CH
12 INT 21H
```



Superscalar Execution (aka pipelining)

- ISA instructions are NOT executed one after the other!
 - Nor do they take one cycle to complete!
 - Instead, they are broken down to microarchitecture instructions
 - Each can be executed by a different component in the CPU
- Thus, we parallelize/pipeline them
 - What if there is a branch?



https://en.wikipedia.org/wiki/Superscalar_processor



Speculative Execution

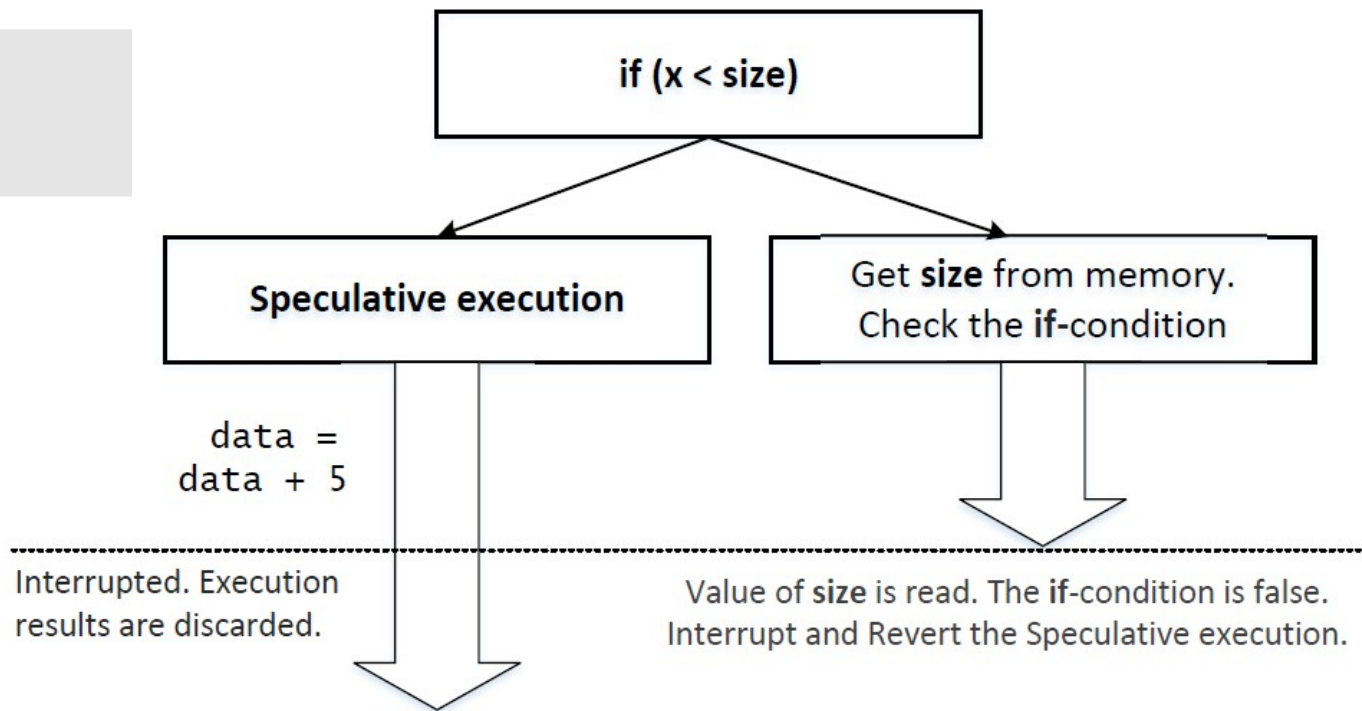
- The CPU guesses/learns the path the program will follow and starts executing it by putting it into the pipeline
- If it made a wrong guess, it flushes the pipe and starts from the right point undoing all changes it performed.

```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



Speculative Execution

```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



Processor manufacturers forgot about the Cache

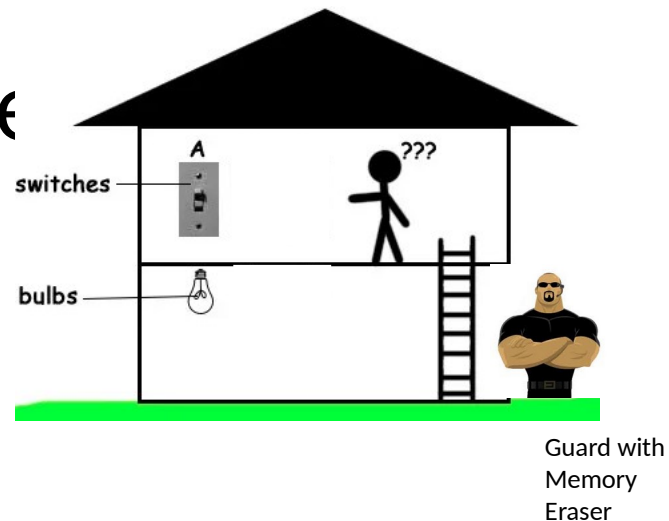


The Meltdown attack

- Attacker is a user program and is therefore in the user space.
- The attacker's motivation is to get hold of some confidential information in the kernel space.
 - The user space/kernel space separation using the on board hardware prevents an attacker from doing it directly.
 - The meltdown attack allows the attacker to do this using a side channel



From Lights to CPU Cache



Room = Kernel Memory

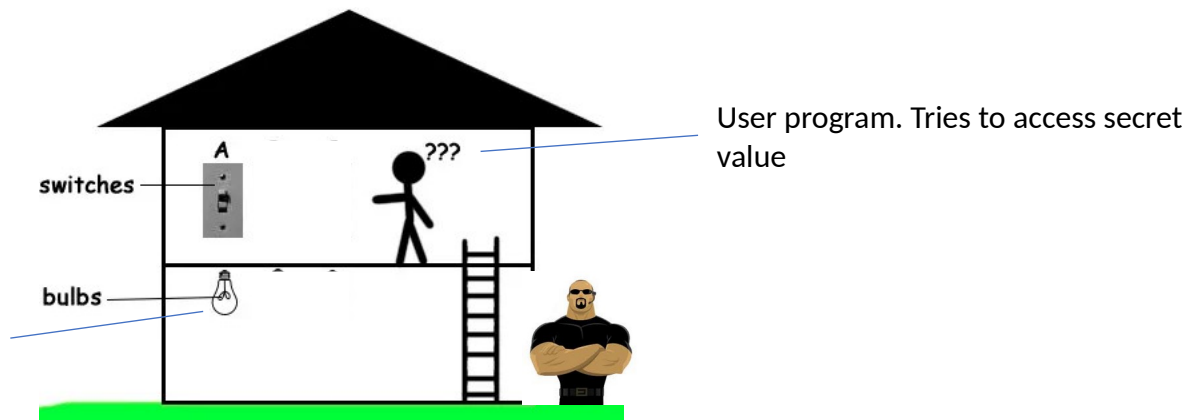
Secret number = Confidential kernel information

Guard = Access Control System. Uses the hardware to check if access is allowed.



From Lights to CPU Cache

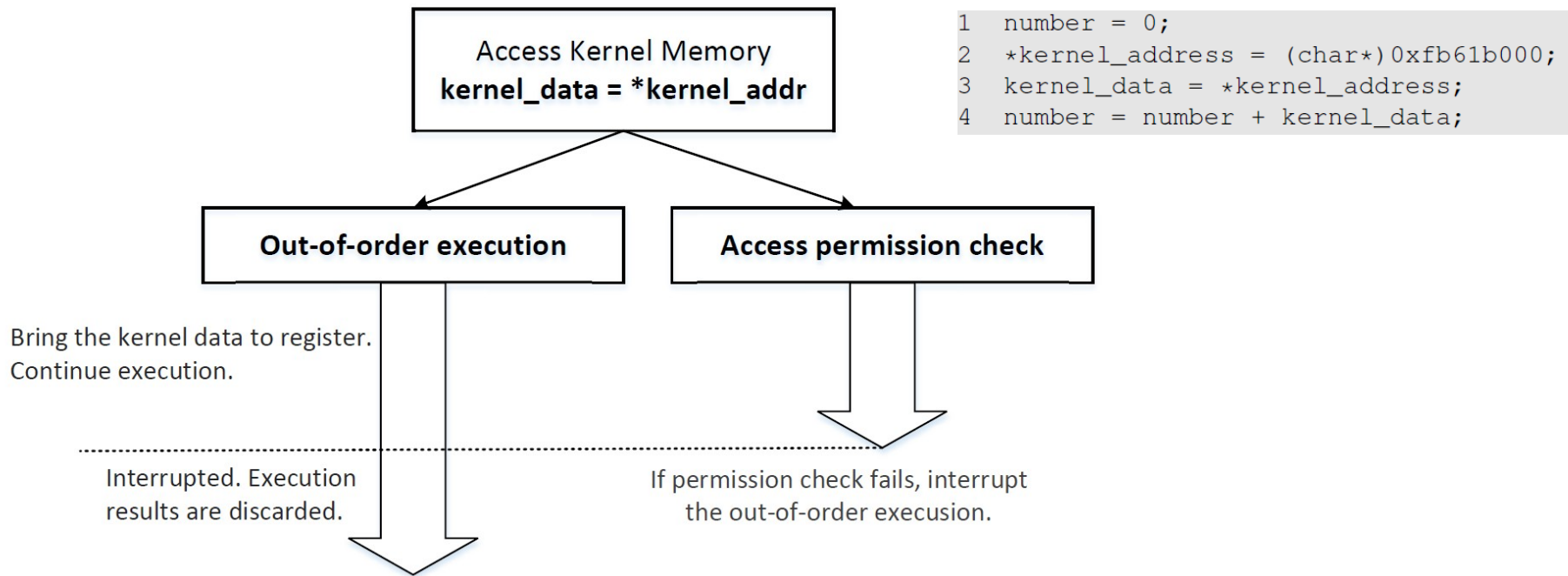
The information accessed by the program is now hot in the cache. The program can now read the cache and therefore the secret value



Access control system:
Checks Access but
speculative execution
causes delay, while the
program instructions are
executed



Speculative/Out-of-order Execution



Speculative/Out-of-order Execution

```
1 data = 0;  
2 if (x < size) {  
3     data = data + 5;  
4 }
```



The Spectre attack

- Makes use of speculative execution's drawback too
- Turns out you can (potentially) play this trick every time there is a condition in the code
- CPUs try to make an educated guess of which execution path is likely to be taken.
 - They do this by looking at which path was historically taken
- A CPU can be fooled into taking the incorrect execution path by training it with the correct execution path
- Eventually the CPU will correct itself but any data in the CPU cache as a result of the incorrect execution will not be erased



The Spectre attack

- Many access control decisions are taken using conditional code

```
if ( read_permission == 1)
    allow read
```

- Spectre class of attacks exploits speculative execution to get access to resources without having the required permission

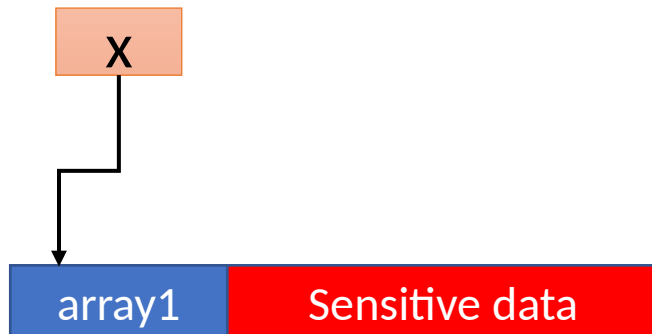


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

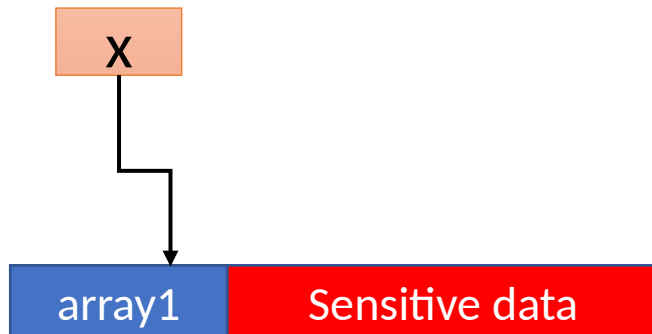


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

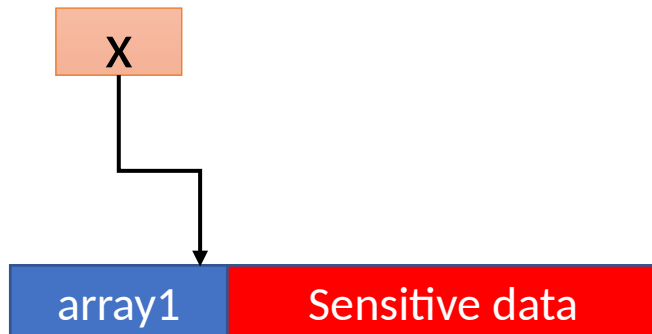


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

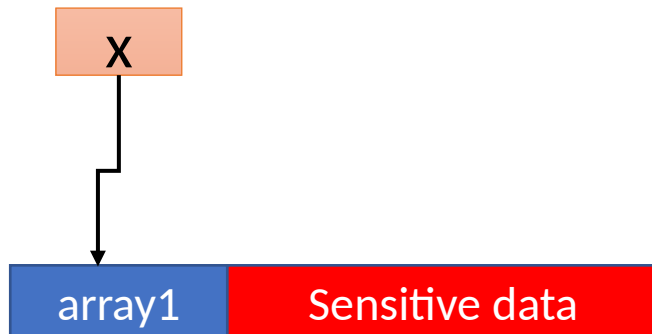


Spectre Overview

Phase 1

Mis-Train the CPU to speculatively execute the true branch of an if-statement.

Pick repeated valid values for x



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

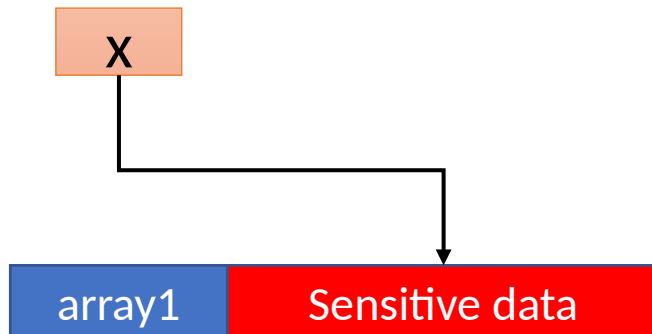
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview

Phase 2-a

Make x much larger than the length of the array and try to read beyond it!



```
for(...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

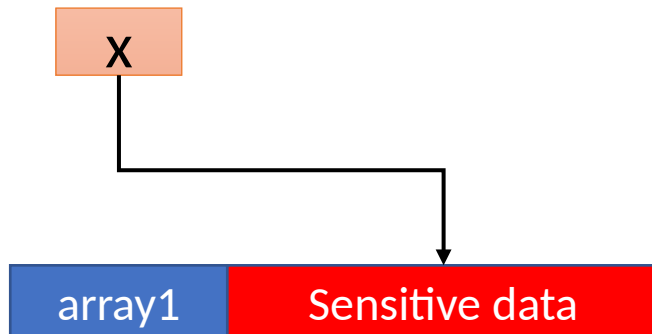
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview

Phase 2-b

The CPU is tricked and starts speculatively executing parts of the true-branch without having completed the actual check yet!



```
for(...) {  
    ...  
    → if (x < array1_size) {  
        → y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>



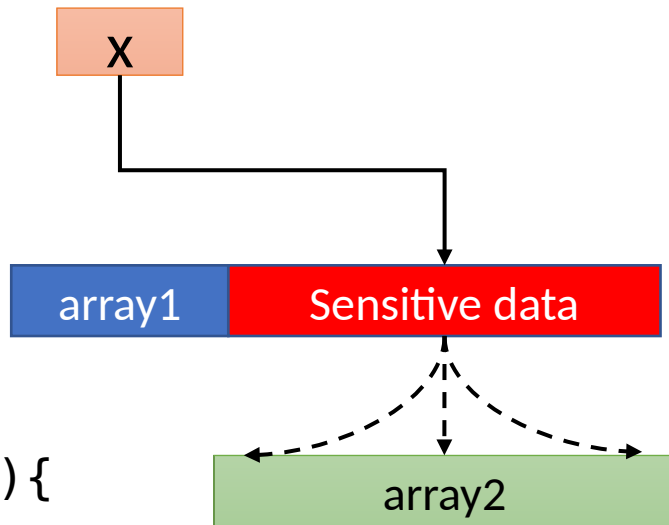
Spectre Phase 2-c

An index for array2 is
calculated:

$\text{array1}[x] * 4096$

Remember, $\text{array1}[x]$ is sensitive data now! The
index for array2 is sensitive too

E.g., if $\text{array1}[x]$ is 0, the
calculated index is 0. If it is 1,
the index is 4096. If 2, 8192
and so on...



```
for(...) {  
    ...  
    → if (x < array1_size) {  
        → y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

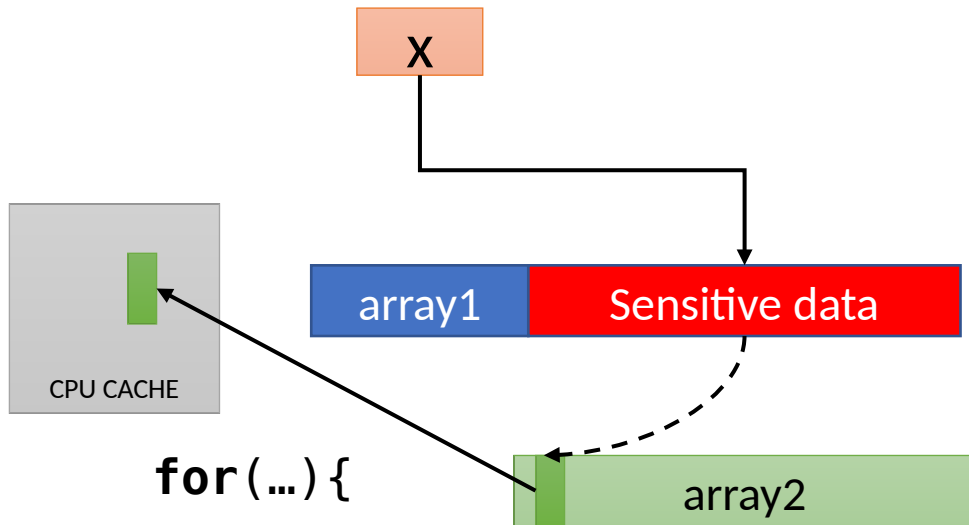


Spectre Overview

Phase 2-d

An index for array2 is calculated:
 $\text{array1}[x] * 4096$

The memory from array2 at that index is brought into the cache!



```
for (...) {  
    ...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```

Based on <https://spectreattack.com/spectre.pdf>

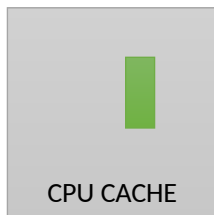


Spectre Overview Phase 2-e

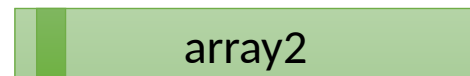
The CPU realizes its speculative execution failed and stops trying to execute the true-branch.

It unrolls all changes it performed.

But leaves the cache as is!!!



```
for (...) {
```



```
...  
    if (x < array1_size) {  
        y = array2[array1[x] * 4096];  
    }  
    ...  
}
```



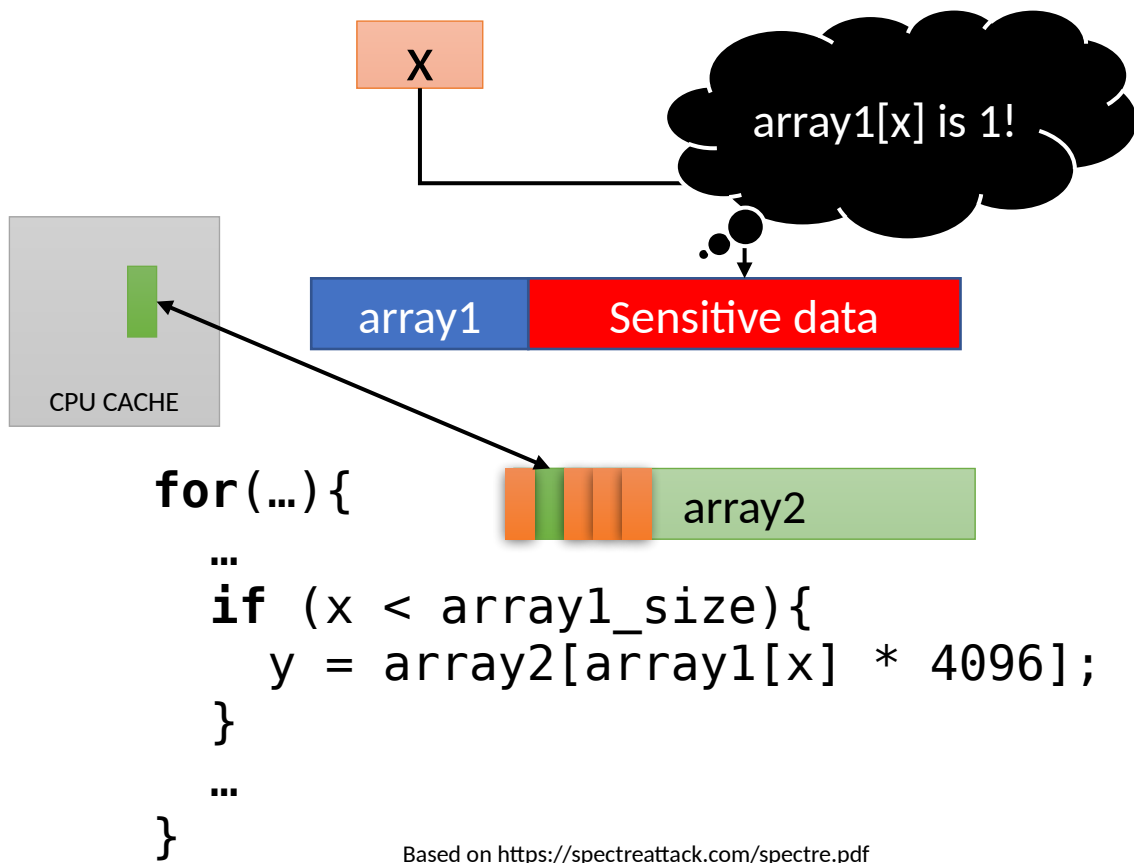
Based on <https://spectreattack.com/spectre.pdf>



Spectre Overview Phase 3

The attacker measures the time it takes to load a byte from array2 at a cache-line distance.

They will all be slow except one! The one already in the cache! Its position reveals the value of array1[x]



Based on <https://spectreattack.com/spectre.pdf>



Spectre Attacks: Exploiting Speculative Execution

Paul Kocher¹, Jann Horn², Anders Fogh³, Daniel Genkin⁴,
Daniel Gruss⁵, Werner Haas⁵, Mike Hamburg⁷, Moritz Lipp⁵,
Stefan Mangard⁵, Thomas Prescher⁶, Michael Schwarz⁵, Yuval Yarom⁸

¹ Independent (www.paulkocher.com), ² Google Project Zero,

³ G DATA Advanced Analytics, ⁴ University of Pennsylvania and University of Maryland,

⁵ Graz University of Technology, ⁶ Cyberus Technology,

⁷ Rambus, Cryptography Research Division, ⁸ University of Adelaide and Data61

Abstract—Modern processors use branch prediction and speculative execution to maximize performance. For example, if the destination of a branch depends on a memory value that is in the process of being read, CPUs will try to guess the destination and attempt to execute ahead. When the memory value finally arrives, the CPU either discards or commits the speculative computation. Speculative logic is unfaithful in how it executes, can access the victim's memory and registers, and can perform operations with measurable side effects.

Spectre attacks involve inducing a victim to speculatively perform operations that would not occur during correct program execution and which leak the victim's confidential information via a side channel to the adversary. This paper describes practical attacks that combine methodology from side channel attacks, fault attacks, and return-oriented programming that can read arbitrary memory from the victim's process. More broadly, the paper shows that speculative execution implementations violate the security assumptions underpinning numerous software security mechanisms, including operating system process separation, containerization, just-in-time (JIT) compilation, and countermeasures to cache timing and side-channel attacks. These attacks represent a serious threat to actual systems since vulnerable speculative execution capabilities are found in microprocessors from Intel, AMD, and ARM that are used in billions of devices.

While makeshift processor-specific countermeasures are possible in some cases, sound solutions will require fixes to processor designs as well as updates to instruction set architectures (ISAs) to give hardware architects and software developers a common understanding as to what computation state CPU implementations are (and are not) permitted to leak.

leverage hardware vulnerabilities to leak sensitive information. Attacks of the latter type include microarchitectural attacks exploiting cache timing [8, 30, 48, 52, 55, 69, 74], branch prediction history [1, 2], branch target buffers [14, 44] or open DRAM rows [56]. Software-based techniques have also been used to mount fault attacks that alter physical memory [39] or internal CPU values [65].

Several microarchitectural design techniques have facilitated the increase in processor speed over the past decades. One such advancement is speculative execution, which is widely used to increase performance and involves having the CPU guess likely future execution directions and prematurely execute instructions on these paths. More specifically, consider an example where the program's control flow depends on an uncached value located in external physical memory. As this memory is much slower than the CPU, it often takes several hundred clock cycles before the value becomes known. Rather than wasting these cycles by idling, the CPU attempts to guess the direction of control flow, saves a checkpoint of its register state, and proceeds to speculatively execute the program on the guessed path. When the value eventually arrives from memory, the CPU checks the correctness of its initial guess. If the guess was wrong, the CPU discards the incorrect speculative execution by reverting the register state back to the stored checkpoint, resulting in performance comparable to idling. However, if the guess was correct, the speculative execution

Kocher, Paul, Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. "Spectre attacks: Exploiting speculative execution." *arXiv preprint arXiv:1801.01203* (2018).



COMPX304-23A

Advanced Networking and Cybersecurity

Week 11
Secure Software Development

Vimal Kumar

Slides: Vimal Kumar, Chris Anderson, Panos Patros



What to do?

- We've seen examples of vulnerabilities and attacks
- Various levels
 - Users/People, Software, Operating Systems, Hardware
- Need for systematic approach for secure software
 - Secure Software Development and Architecture
 - Security Testing



Secure Coding Practices I

- Do not trust User Input
 - check nulls, types, reserved words, expected formats
- Don't ignore compiler warnings
- Keep it simple!
- Default deny

```
if isUserAuthorised:  
    authorizedAction()
```



```
if !isUserAuthorised:  
    return;  
  
authorizedAction()
```



Secure Coding Practices II

- Principle of least privilege
- Sanitize data sent to other systems
- Remember, multitier sanitization!!
 - Validate sanitization functions you use!

- People copy random code from the Internet... ☐

```
input = "<scr<script>ipt>nastyCode() </scr<script>ipt>"
```

```
print singleLevelSanitize(input)
```

```
<script>nastyCode()</script>
```

```
print multiLevelSanitize(input)
```

```
nastyCode()
```



Secure Coding Practices III

- Reuse existing **validated** modules, frameworks, ...
 - Don't write your own security code
 - Don't use low level languages unless needed
- Check for integer overflows
- Don't compare floats with strict equality:

```
float a, b;  
if(abs(a-b)>=threshold) //Good ☐  
if(a==b) //Bad ☐
```
- Check string lengths (buffer overflows)



Requirements

- Core CIA (Confidentiality, Integrity, Availability)
- Important extensions
 - Authentication
 - Identity verification
 - Nonrepudiation
 - Message ownership
 - Authorization
 - Privilege granting



Threat Modelling

- Threat Modelling using attack trees

- Root

- Successful Attack

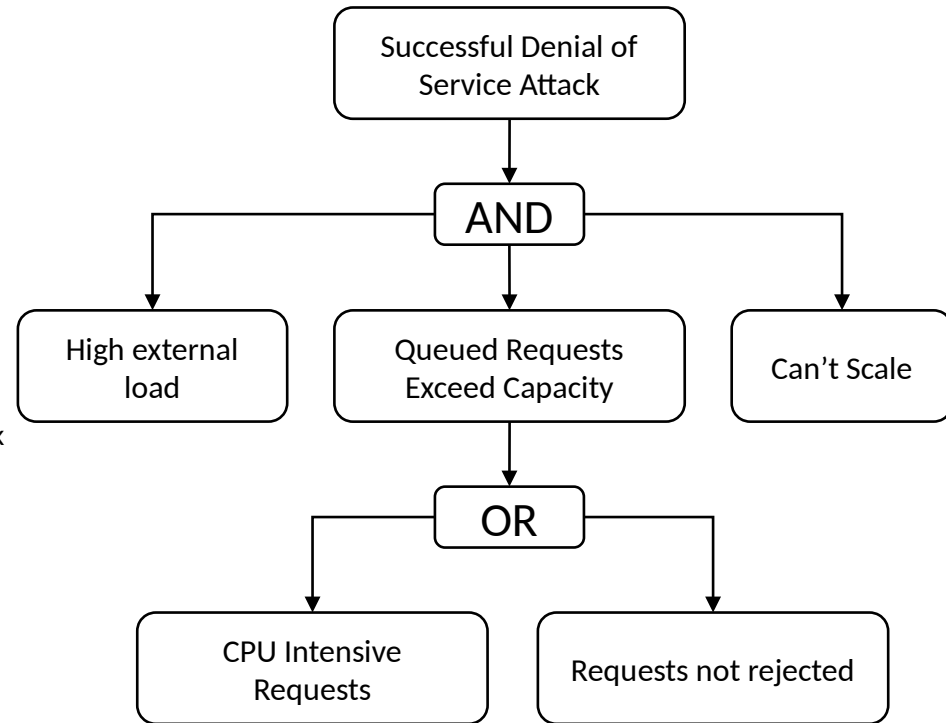
- Nodes

- Conditions required for the success of the attack

- Logical operators

- Joining the conditions

- Allows one to think about how systems can be attacked and how one can prevent the conditions that enable an attack



Security Testing

- Effective QA
 - Test plans, security requirements & regression testing
 - Penetration tests
 - Source code auditing
 - Independent security reviews
 - Automated toolchains
 - Bug tracking tools
 - Secure coding standards

